

Enterprise AI Governance Platform

Technical documentation set

20 documents

Documentation set: Enterprise AI Governance

Version 1, 2026-07-28. Written against the code as it stands, not as planned.

#	Document	What it answers
01	Requirements	What the system must do, numbered and testable
02	High-Level Design	The components, the data flow, the tech choices
03	Low-Level Design	Inside each component: nodes, agents, packs, state
04	Data Model	The asset record, findings, the estate, the packs
05	API Specification	Every endpoint, its inputs, outputs, and status codes
06	Decision Log (ADRs)	Each architectural decision, its options and consequences
07	Configuration and Secrets	Every environment variable and where its real value lives
08	Runbook	Start, stop, seed, recover, and what to do when it breaks
09	Test Plan and Report	What is tested, how, and the last measured result
10	Benchmark Report	Measured tier accuracy, recall, precision, audit completeness
11	Non-Functional Requirements	Performance, availability, privacy targets with numbers
12	Security Review	Threat model: what an attacker could try and what stops them
13	Privacy and Data Handling	What data is touched and where it goes
14	Model Card	Which model, its limits, known failure modes
15	Risk Register	What could sink the project and what mitigates it
16	Traceability Matrix	Requirement to code to test, so coverage is provable
17	User Guide	How a reviewer operates the control tower
18	Release Notes	What changed, in order, and what it broke
19	Handover	Everything the next owner needs

Also in this folder: the client requirement PDF, and [demo-script.md](#) for the four-beat walkthrough.

01. Requirements

Version 1, 2026-07-28. Authoritative source: the client requirement PDF in this folder, plus the project plan in the planning repository. Evidence for every status is in [16-traceability-matrix.md](#).

1. Problem

A company ends up with dozens of AI models and agents scattered across teams. Nobody holds a single list of what exists, who owns it, what data it touches, which rules apply to it, or whether anyone ever checked. When a regulator or an executive asks, the answer takes weeks of chasing and is out of date on arrival.

2. What this product is

A control tower. It keeps a central inventory of every AI asset in the company, judges each one against a **swappable rulebook**, rolls the findings into a risk tier, routes flags to a human reviewer, sweeps the estate on a schedule, and keeps a tamper-evident audit trail for executive and regulatory reporting.

Nothing is auto-blocked. A flag is a work item for a person, never an automatic shutdown. That is a deliberate product decision, not a limitation.

3. Functional requirements

ID	Requirement	Status
FR-1	Register an AI asset from a plain-language description and turn it into a canonical record	MET
FR-2	Hold a central inventory of the estate, queryable and filterable	MET, 185 seeded assets
FR-3	Judge each asset against a policy pack and a regulatory framework pack, both of which are data rather than code	MET
FR-4	Inspect assets across five dimensions: policy compliance, risk, data governance, responsible AI, and security and third parties	MET, five inspectors in parallel
FR-5	Choose which inspectors apply to a given asset rather than always running all five	MET, an orchestrator agent decides, defaulting to all five
FR-6	Assign a regulatory risk tier to each asset	MET, EU AI Act tiers
FR-7	Roll findings into a risk level and score deterministically, never by a model	MET
FR-8	Every finding must carry the control it pins to, a severity, a plain-English line, the evidence, and a remediation	MET, enforced by validation

ID	Requirement	Status
FR-9	Route a flag to a human reviewer, with no automatic blocking	MET
FR-9a	Record what the reviewer concluded: approve a finding as real, or override it with a mandatory reason, and put that verdict in the audit chain	MET
FR-9b	Track remediation as work: every finding can carry an owner, a due date and a status (open , in_progress , awaiting_evidence , closed , dismissed), workable from a board with drag and drop, with every change in the audit chain. An approved finding stays open, because approval starts remediation rather than ending it	MET 2026-07-28
FR-9c	Attach evidence to a finding: the file that proves the remediation happened, downloadable later by an auditor, with the upload recorded in the audit chain	MET 2026-07-29
FR-10	Swap the active rulebook and re-score the estate against the new one	MET, deterministic re-scoring, no model calls
FR-11	Sweep the estate on demand: monitoring, regulatory intelligence, and audit reporting	MET
FR-12	Produce an executive brief over the estate	MET
FR-13	Keep a tamper-evident audit trail per asset, readable through the API	MET, verified against a forged database entry
FR-14	Administer users: an administrator creates reviewer accounts; no open signup	MET

4. Non-functional requirements

ID	Requirement	Target	Status
NFR-1	Rules are data, loaded at call time, so a pack can be swapped without a code change or a restart	Config, not code	MET
NFR-2	Asset descriptions never reach a third-party model	Zero exceptions	MET by construction: one lane, no cloud client
NFR-3	An assessment completes unattended	minutes	MET, measured about 220 seconds on a warm lane
NFR-4	A stuck agent cannot hang an assessment	Per-node guard	MET
NFR-5	The audit trail detects tampering	Detection, not prevention	MET
NFR-6	Deterministic parts stay deterministic: risk roll-up and pack matching are plain code	Reproducible	MET

ID	Requirement	Target	Status
NFR-7	Synthetic estate only, secrets outside the repository	No real corporate data	MET
NFR-8	Reuse the skeleton from the two earlier systems	Module-level reuse	MET

5. Out of scope

No agent or model runtime integration, so nothing is measured by watching a live system. No automatic enforcement or shutdown. No procurement, no vendor management, no incident response. No jurisdiction-specific legal advice: the framework pack encodes rules, it does not interpret them.

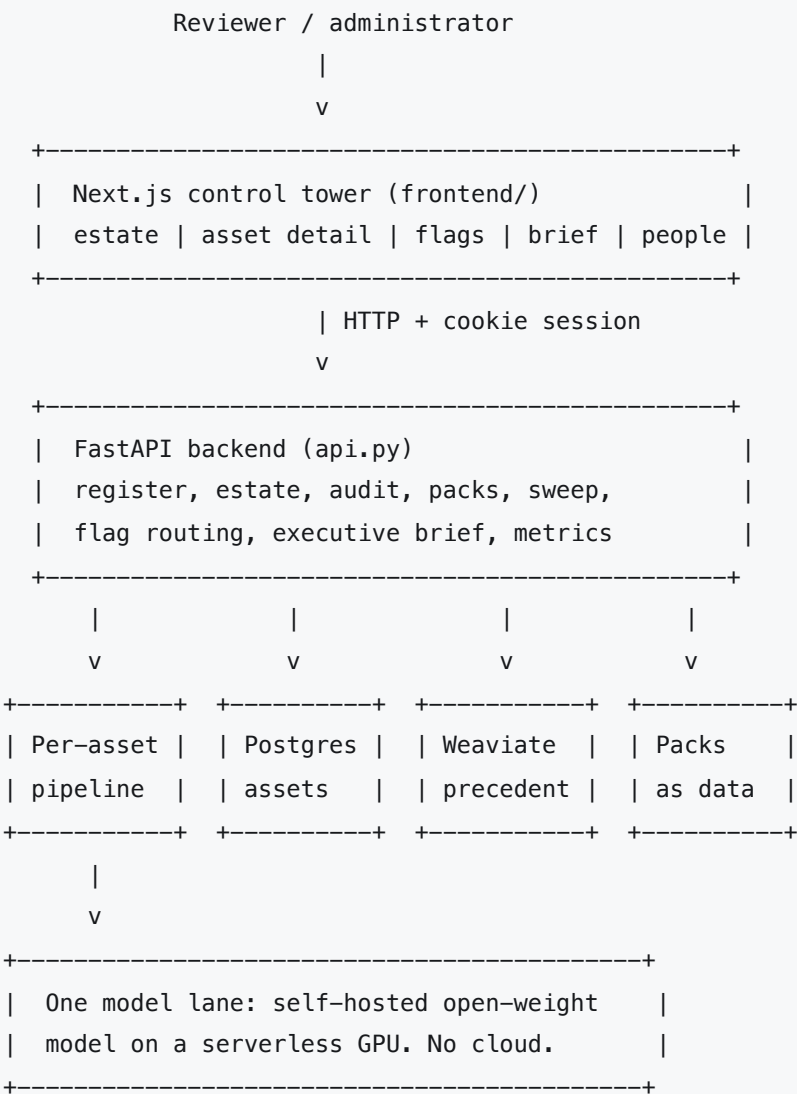
6. Known requirement gaps

- **Tier accuracy is 73 percent** on the labelled set, with one asset under-rated. Tiering is the highest-stakes judgement the system makes.
- **Flag precision is 46 percent:** roughly half the flags raised were not in the answer key. Some are genuine, some are noise, and they are not separable without a second reviewer.
- The pack swap re-scores policy findings deterministically, but **regulatory tiers do not move in the re-score**, because tiering needs the model. This is stated on the demonstration path rather than glossed over.

02. High-Level Design

Version 1, 2026-07-28.

1. The shape of the system



2. The per-asset pipeline



Node	Model call	Responsibility
inventory	yes	Turn a plain-language registration into a canonical asset record
orchestrate	yes	Decide which of the five inspectors actually apply to this asset
Five inspectors	yes, with tools	Findings against the active packs, run as one parallel step
fan_in	no	Validate findings, lower-case the tier the model wrote, roll up risk deterministically
decide	no	compliant or flagged . Plain code, and never an automatic block

3. Beyond one asset: the estate

Three things operate over the whole estate rather than a single asset:

Capability	What it does
Pack swap	Activating a different policy pack re-scores every policy finding across the estate deterministically, with no model calls, by matching each rule's machine-readable applicability against each asset
Sweep	Three additional agents run over a slice of the estate: model monitoring, regulatory intelligence, and audit reporting
Executive brief	A roll-up over the estate for leadership

The pack swap is the demonstration centrepiece and it costs nothing to run, because it is deterministic matching rather than reasoning.

4. Components

Component	Where	Responsibility
Control tower	frontend/	Estate list, asset detail, flags, brief, people administration
API	api.py	Registration, estate queries, audit, packs, sweep, flag routing, metrics
Pipeline	app/graph.py	Nine-node graph with a conditional fan-out
Agents	app/agents.py , app/agents_base.py	Inventory, orchestrator, five inspectors, three sweep agents
Packs	app/packs.py , data/policy_packs , data/framework_packs	Rules as data, loaded at call time
Tools	app/tools.py	Seven read-only lookups
Model lane	app/router.py	One endpoint, one lock, one retry

Component	Where	Responsibility
System of record	<code>app/store.py</code> on Postgres	One row per asset plus the full state
Precedent	<code>app/precedent.py</code> on Weaviate	Prior assessments retrieved by similarity
Sweep	<code>app/sweep.py</code>	Estate-wide runs, re-scoring, estate metrics
Audit	<code>app/audit.py</code>	Hash chain per asset

5. Flow of one registration

1. A reviewer pastes a paragraph describing an AI system.
2. The asset is parked as processing immediately, so the estate has a card to show.
3. Inventory canonicalises the description into a record: name, type, owner, lifecycle, data touched, and so on.
4. The orchestrator picks the applicable inspectors.
5. Those inspectors run in parallel against the active packs, using read-only tools.
6. Fan-in validates and rolls up; decide marks the asset compliant or flagged.
7. Flags are routed to a person. Nothing is blocked automatically.

Measured live: a registration took about 220 seconds on a warm lane, produced five findings across four inspectors, assigned the correct tier, and left an intact eight-entry audit chain.

6. Technology choices

The stack matches the two sibling systems: Python and FastAPI, LangGraph, Postgres, Weaviate, Next.js, and a single self-hosted model lane on serverless GPU. The one design idea that is unique here is **packs as data**: the rulebook is a file, matched by a machine-readable applicability spec, so swapping it is configuration rather than a release.

03. Low-Level Design

Version 1, 2026-07-28.

1. The state object

Key	Written by	Holds
asset_id	caller	Identity
status	several	processing , assessed , flagged , error
stage	every node	Narration: intake, orchestrating, inspecting, rolling up, done
description	caller	The messy registration text
asset	inventory	The canonical record
applicable_inspectors	orchestrate	Which inspectors apply, defaulting to all five
findings_raw	five inspectors, append-only	Findings before validation
inspector_reports	five inspectors, append-only	Per inspector: ok or failed, plus a note
risk_tier	risk assessment, lower-cased at fan-in	The regulatory tier
risk	fan-in	Level, score, why
decision	decide	compliant or flagged
audit	every node	The hash chain, append-only

The casing trap: the tier comes back written the way the model wrote it, capitalised or not. It is lower-cased once, at fan-in, and every reader downstream relies on that. Any new reader must not assume the raw value.

2. The finding shape

A finding carries a finding id, the inspector, the **control id** it pins to (a rule from the policy pack or a control from the framework pack), a severity, a plain-English line, the evidence that triggered it, a remediation, and a status. Anything missing a required field is discarded before display and counted.

Since 2026-07-28 a finding also carries its remediation state: an owner , a due_at date, an evidence_files list, and a five-word status vocabulary (open , in_progress , awaiting_evidence , closed , dismissed). These live inside the same JSONB shape, so no migration ran and findings written earlier read as unassigned and open. store.list_findings() flattens findings across the estate the same way the open-finding counter always has, and PATCH /flags/{finding_id} does the read-modify-write; both follow the existing single-table pattern rather than normalising into a findings table (a five-hour job instead of twelve, at the accepted cost of last-write-wins on concurrent edits to one asset).

`evidence_files` is the one field on that list that is a mirror rather than the record itself. Since 2026-07-29 the files behind it live in a dedicated `evidence` table (ADR-014) because the asset blob is read whole on every estate view and bytes inside it would tax all of those reads; the finding keeps only each file's id, name, size and uploader so a board card can show a count without a second query. `store.add_evidence` writes both halves in one request path, and there is no delete route.

Pinning every finding to a control id is what makes the audit trail defensible: a reviewer can ask which rule this came from and get an answer.

3. Packs as data

Two kinds of pack live as files rather than code: policy packs (a company's own rules) and framework packs (a regulatory framework's controls).

- Packs are **loaded at call time**, anchored to the code rather than the working directory, so changing the active pack needs no restart and no redeployment.
- Each policy rule carries a machine-readable applicability spec. Plain code decides whether a rule fires for an asset by matching that spec against the asset's fields.
- Because matching is deterministic, **swapping the active pack re-scores the whole estate without a single model call**. Measured: one pack produced 194 findings across 111 assets, the other 147 across 114, with 86 assets re-scored, and every audit chain left intact.

The honest limit, stated on the demonstration path: regulatory **tiers** do not move in a re-score, because assigning a tier needs the model.

4. Agents

Agent	Role
<code>inventory</code>	Plain-language description to a canonical record
<code>orchestrate</code>	Which inspectors apply to this asset
<code>policy_compliance</code>	Breaches of the active policy pack
<code>risk_assessment</code>	The regulatory risk tier and risk findings
<code>data_governance</code>	What data is touched, on what basis, with what retention
<code>responsible_ai</code>	Fairness, bias testing, transparency, human oversight
<code>security_third_party</code>	Model supply chain, third-party services, access
<code>model_monitoring</code>	Sweep only: drift and performance signals
<code>regulatory_intel</code>	Sweep only: what changed in the rules
<code>audit_reporting</code>	Sweep only: reporting over the estate

Each inspector is a reasoning loop with a step ceiling and seven read-only tools: read the registry, read the policy pack, read the framework pack, search precedent, read an asset's history, read its audit trail, and

read the active pack. All are read-only by design: a governance system that could change what it governs would not be a governance system.

5. Guards and failure handling

Every node is wrapped with a bounded wall-clock and one retry, recording failure into the state rather than raising.

Failure	Behaviour
One inspector fails both attempts	Marked failed with a note; the others still produce findings
The orchestrator finds nothing applicable	The graph ends cleanly rather than running five inspectors for nothing
Model output carries a second JSON object	The parser takes the first complete object
Parallel inspectors arrive together	A client-side lock serialises them, and the platform-side single-container cap prevents a second billed GPU. A vLLM swap removed the lock on 2026-07-28 and was rolled back on 2026-07-29 (ADR-009 amendment)
A stray server error from the lane	One retry, because a container swap surfaces exactly this way

6. Deterministic by choice

Three things are plain code and must stay that way, because they are what makes the benchmark meaningful:

1. **The risk roll-up.** Agents never write it.
2. **Pack applicability matching.** A rule fires or it does not.
3. **The decision.** Compliant or flagged, from the findings, never a model's opinion, and never an automatic block.

7. Module map

Module	Responsibility
api.py	Endpoints, authentication, background processing
app/graph.py	Nine nodes, guards, the conditional fan-out
app/agents.py	Ten agent prompts and the inspector runner
app/agents_base.py	The reasoning loop and the JSON parser
app/packs.py	Pack loading and applicability matching
app/sweep.py	Estate sweep, deterministic re-scoring, estate metrics

Module	Responsibility
app/tools.py	The seven read-only tools
app/router.py	The single model lane
app/store.py	Postgres access
app/precedent.py	Weaviate collection
app/state.py	Shapes, validation, risk roll-up
app/audit.py	Hash chain
app/users.py	Accounts, roles, sessions

04. Data Model

Version 1, 2026-07-28. Three kinds of storage: Postgres for the estate, Weaviate for precedent, and plain files for the rulebooks.

1. Postgres, database `governance`

`assets`

Column	Type	Meaning
<code>asset_id</code>	TEXT, primary key	Identity, for example <code>AI-0042</code>
<code>name</code> , <code>type</code> , <code>owner</code> , <code>lifecycle</code>	TEXT	Denormalised from the canonical record, for estate filtering
<code>status</code>	TEXT	<code>processing</code> , <code>assessed</code> , <code>flagged</code> , <code>error</code>
<code>stage</code>	TEXT	Narration while an assessment runs
<code>risk_level</code>	TEXT	<code>high</code> , <code>medium</code> , <code>low</code> , rolled up deterministically
<code>risk_tier</code>	TEXT	The regulatory tier, lower-cased at fan-in
<code>source</code>	TEXT	<code>seed</code> for the synthetic estate, <code>live</code> for one registered through the tower
<code>state</code>	JSONB	The whole assessment: record, findings, reports, risk, decision, audit chain
<code>created_at</code>	TIMESTAMPTZ	Registration time

The `source` column matters more than it looks: it is how a genuinely live registration is told apart from the seeded estate during a demonstration.

`evidence`

Column	Type	Meaning
<code>id</code>	SERIAL, primary key	Download handle
<code>finding_id</code>	TEXT, indexed	The finding this proves work on
<code>asset_id</code>	TEXT	Denormalised so evidence can be swept per asset without opening any blob
<code>filename</code> , <code>content_type</code>	TEXT	As uploaded
<code>size</code>	INT	Bytes, computed server-side rather than trusted from the client
<code>data</code>	BYTEA	The file itself

Column	Type	Meaning
uploaded_by	TEXT	The account's email
created_at	TIMESTAMPTZ	Upload time

Why this is a table when the four remediation fields were not. Those were small scalars, so putting them in the existing JSONB avoided a migration and cost nothing. Bytes are different: the asset blob is read in full on every estate view, and a few screenshots inside it would slow down every one of those reads for the sake of a column almost nobody opens. So the bytes get a table, and only the file's *metadata* is mirrored onto the finding, which lets the board show a count per card without a second query. **The table is the source of truth for content; the mirror is a display convenience.**

Accounts

Roles are `reviewer` and `admin`. **No open signup**; an administrator creates every account.

2. The canonical asset record

Produced by the inventory agent from a paragraph of prose: name, type, owner, lifecycle stage, purpose, the data it touches, who it affects, whether a human reviews its output, and the like. This canonicalisation is the whole reason a registration can be a paragraph rather than a form.

3. The finding shape

Field	Meaning
finding_id	Stable id, carrying the asset and the inspector
inspector	Which of the five raised it
control_id	The policy rule or framework control it pins to
severity	high, medium or low
plain	What it means in everyday words
evidence	The fact in the record that triggered it
remediation	What to do about it
status	One of <code>open</code> , <code>in_progress</code> , <code>awaiting_evidence</code> , <code>closed</code> , <code>dismissed</code> . Only an override can set <code>dismissed</code> , and a dismissed finding cannot be moved again
owner	Email of the person remediating it, or null for unassigned
due_at	ISO date the remediation is due, or null
evidence_files	List of attached proof references, empty until evidence upload ships
routed_to	The team it was routed to, if it has been
review	The reviewer's verdict, once given: verdict, reason, who, and when

The four remediation fields (`status` beyond its original two words, `owner` , `due_at` , `evidence_files`) were added inside the existing JSONB shape on 2026-07-28, so **no migration ran**: a finding written before then simply reads as unassigned and `open` , which is exactly what it means. One consequence is accepted and documented rather than solved: a finding update is a read-modify-write of one asset's JSON document, so two people editing findings on the same asset at the same moment can clobber each other. Acceptable for two reviewers on a demo estate; row locking would cost more than it buys this week.

A finding missing any required field is discarded before it reaches a reviewer, and the drop is counted.

Why an approval leaves the status alone. Approving a finding confirms it is real, so the remediation work still exists and it must keep counting as open. Only an override closes it. That keeps the estate's open-finding counts meaning "work outstanding" rather than "not yet looked at".

4. Packs, as data

Kind	Location	Contents
Policy packs	<code>data/policy_packs</code>	A company's own rules, each with a machine-readable applicability spec
Framework packs	<code>data/framework_packs</code>	A regulatory framework's controls
Estate	<code>data/estate</code>	The synthetic asset corpus

Two policy packs and one framework pack ship with the system, which is what makes the swap demonstrable: activating the other pack re-scores the estate deterministically.

5. The seeded estate

185 synthetic assets, plus a 15-asset labelled answer key with **zero overlap** with the rest, used for the benchmark. Seeded assets carry deliberately **empty audit chains**, so that any chain you see was produced by a real run rather than by the seeder.

Since 2026-07-28 the seeder also walks findings through a fixed index-based cycle of owners, due dates and statuses, so the remediation board opens populated rather than empty (at full scale roughly: 120 open, 79 in progress, 39 awaiting evidence, 39 closed, 79 unassigned, 39 overdue). The cycle is deterministic, so re-seeding is reproducible; it adds no audit entries, so the empty-chain rule above still holds; and it never overwrites a dismissed finding.

6. Weaviate, collection for precedent

Prior assessments, retrievable by similarity so a later asset can cite an earlier ruling. Embeddings are computed locally, with the cache pinned inside the repository after a temporary-directory cache corrupted itself.

7. Audit chain

Per asset, one entry per pipeline step, each sealed against the previous. Read through the audit endpoint, which verifies on read and reports the first broken index. Tested by forging an entry directly in Postgres: the break was reported at the exact index.

8. Retention

Nothing is deleted. Assets, assessments, findings and audit chains are kept indefinitely, which suits a governance record. There is no purge and no deletion endpoint. The whole estate is synthetic.

05. API Specification

Version 1, 2026-07-28. Base URL `http://localhost:8000` . A live version is served at `/docs` .

1. Authentication and roles

Signed cookie sessions. Two roles: `reviewer` and `admin` . **No open signup**. Every endpoint below except health, configuration and the two first-run setup routes requires a signed-in account.

The one exception to "an administrator creates every account" is the **first-run bootstrap**. A freshly installed system has no accounts, therefore no administrator, therefore no way to create the first one. `GET /auth/needs-setup` reports that state and `POST /auth/bootstrap` closes it by creating the founding administrator. The bootstrap route refuses with 403 the moment any account exists, so the door opens once and never again. `seed_users.py` remains the scripted route to the same place for a development machine.

Method	Path	Notes
POST	<code>/auth/login</code>	Form credentials, returns a session cookie
GET	<code>/auth/needs-setup</code>	<code>{"needs_setup": true}</code> only while the system holds zero accounts
POST	<code>/auth/bootstrap</code>	Creates the founding administrator from email and password. 403 once any account exists, 409 on a duplicate address
POST	<code>/auth/logout</code>	Ends the session
GET	<code>/users/me</code>	The current account and role

2. Public

Method	Path	Returns
GET	<code>/</code> , <code>/health</code>	Liveness. Answers instantly without touching anything, so it is safe for uptime pings
GET	<code>/healthz</code>	The honest check: opens Postgres and Weaviate and reports each as <code>ok</code> or <code>down</code> , with <code>status ok</code> or <code>degraded</code> . Reports up or down only, never the error text, because the route is unauthenticated and driver errors leak host and user strings
GET	<code>/config</code>	Brand name and tagline

3. Assets

Method	Path	Who	Notes
POST	<code>/assets</code>	reviewer or admin	Register an asset from a plain-language description. Returns immediately with an id; the assessment runs in the background

Method	Path	Who	Notes
GET	/assets	reviewer or admin	The estate, with its status, stage, risk level, tier and open-finding counts
GET	/assets/{id}	reviewer or admin	The full assessment: record, findings, inspector reports, risk, decision
GET	/assets/{id}/audit	reviewer or admin	The hash chain with a verification result

Registration is the hero path: paste a paragraph, watch the card narrate intake, orchestrating, inspecting, rolling up, done.

4. Packs

Method	Path	Who	Notes
GET	/packs	reviewer or admin	Available policy and framework packs, and which are active
POST	/packs/activate	admin	Switch the active pack and re-score the whole estate deterministically , with no model calls

The re-score returns what changed: findings before and after, and how many assets were re-scored. Audit chains survive it intact.

5. Flags and sweep

Method	Path	Who	Notes
POST	/flags/{finding_id}/route	reviewer or admin	Route a flag to a person. Never blocks the asset
POST	/flags/{finding_id}/decision	reviewer or admin	Record what the reviewer concluded: {verdict, reason?} where verdict is approved or overridden

The **decision endpoint** is the other half of routing. Routing says who should look; this says what they found.

Verdict	Effect
approved	The finding is confirmed real. Its status stays open, because the remediation work still has to happen
overridden	The finding is dismissed. Its status becomes dismissed and it drops out of the estate's open-finding counts

Code	When
422	The verdict is neither approved nor overridden

Code	When
422	An override was sent with no reason. A dismissal without a reason is exactly what an auditor objects to, so it is refused
403	The deciding reviewer owns this finding's remediation. Maker-checker: the person doing the work cannot also sign it off
404	No asset for that finding id, or the finding is not on its asset
409	The flag has already been decided; the first verdict stands

Either verdict appends an entry to that asset's hash chain naming the finding, the verdict, the reviewer and the reason, so the human decision sits inside the tamper-evident record rather than beside it.

5a. Remediation

The remediation queue is the work-tracking half of governance: every confirmed finding is somebody's job, with an owner, a deadline and a status.

Method	Path	Who	Notes
GET	/remediation	reviewer or admin	Flattens findings across the whole estate, each with its asset context (asset id, name, tier, routed team). Returns the rows plus per-status counts and overdue and unassigned totals
PATCH	/flags/{finding_id}	reviewer or admin	Sets any of <code>owner</code> , <code>due_at</code> , <code>status</code> . Sending a field set to null clears it; a field not sent is left alone. Appends an entry to that asset's hash chain

Filters on `GET /remediation`, combinable:

Filter	Means
<code>mine=true</code>	<code>owner</code> equals the signed-in account's email
<code>team=<name></code>	<code>routed_to</code> equals that team name (teams exist only as routing values; accounts carry no team field)
<code>overdue=true</code>	<code>due_at</code> is in the past and the status is neither <code>closed</code> nor <code>dismissed</code>
<code>unassigned=true</code>	<code>owner</code> is null
<code>status=<word></code>	Exact match on one of the five status words

The status vocabulary is `open`, `in_progress`, `awaiting_evidence`, `closed`, `dismissed`. `PATCH` accepts only the first four (the board columns):

Code	When
422	No field sent at all, a status outside the four board columns (the error points at the override path for dismissal), or a malformed <code>due_at</code>
404	No asset for that finding id, or the finding is not on its asset

Code	When
409	The finding is <code>dismissed</code> . A dismissal is a recorded judgement with a reason attached; it cannot be revived by a drag

5b. Evidence

The board has an `awaiting_evidence` column, and this is what it waits for: the file somebody produced to prove the remediation happened. A closed finding an auditor cannot inspect is a claim, not a control.

Method	Path	Who	Notes
POST	<code>/flags/{finding_id}/evidence</code>	reviewer or admin	Multipart upload of one file. Appends an entry to that asset's hash chain and mirrors the file's metadata onto the finding
GET	<code>/flags/{finding_id}/evidence</code>	reviewer or admin	Metadata for every file on that finding. Never returns the bytes
GET	<code>/evidence/{id}</code>	reviewer or admin	Downloads one file as an attachment

Accepted types are PNG, JPEG, GIF, WebP, PDF, plain text and CSV, up to **10 MB**.

Code	When
400	An unsupported content type, or an empty file
413	The file is over the 10 MB ceiling
404	No asset for that finding id, the finding is not on its asset, or no such evidence id
409	The finding is <code>dismissed</code> . Proof of work on a finding that was overridden away would tell a confusing story, so it is refused on the same reasoning as the <code>PATCH</code> route

There is deliberately no delete route. Evidence is an audit record; removing it silently is precisely the operation the hash chain exists to prevent.

The sweep and the estate views:

Method	Path	Who	Notes
POST	<code>/sweep/run</code>	admin	Starts the estate sweep in the background and returns <code>{"state": "started"}</code> immediately (it used to hold the request open for the whole multi-minute run). 409 if one is already running; rate limited to 2 per hour
GET	<code>/sweep/status</code>	reviewer or admin	The current sweep: <code>idle</code> , <code>running</code> , <code>done</code> (with the report), or <code>error</code> (with the reason). The tower polls this
GET	<code>/brief</code>	reviewer or admin	The executive brief over the estate

Method	Path	Who	Notes
GET	/metrics	reviewer or admin	Estate metrics for the dashboard

5c. Tier override and exports

Method	Path	Who	Notes
POST	/assets/{asset_id}/tier	reviewer or admin	Human correction of the AI-assigned risk tier. Body {"tier", "reason"} ; the reason is mandatory (422 without), the tier must be one of the four EU AI Act tiers (422), 409 if the asset already holds that tier. The correction is appended to the asset's hash chain and kept on the asset as tier_override
GET	/export/register.csv	reviewer or admin	The full asset register as CSV, the artifact you hand an external auditor
GET	/export/findings.csv	reviewer or admin	Every finding in the estate as CSV, with its review verdict, reviewer and reason flattened in
GET	/assets/{asset_id}/audit.csv	reviewer or admin	That asset's audit chain as CSV, one row per entry with its hashes, timestamp, actor and an intact flag per row

6. User administration

Method	Path	Who
GET	/users	admin
POST	/users	admin, creates a reviewer or administrator
PATCH	/users/{id}	admin, edits an existing account: its email, its password, or its role. 422 if the role is neither reviewer nor admin , 404 if there is no such account, 409 if the new email is already taken
DELETE	/users/{id}	admin, deactivates

7. Status codes used

Code	Meaning here
200	Success
401	No session
403	Wrong role, for example a reviewer attempting a pack swap
404	Unknown asset, finding or account
409	Duplicate account, or an action that conflicts with the current state

Code	Meaning here
422	Invalid role or malformed input

8. What the API deliberately does not do

There is no endpoint that blocks, disables or quarantines an asset. Routing a flag creates work for a person. That absence is the product's central promise, and it is enforced by there being no such code path at all.

06. Decision Log (Architecture Decision Records)

Version 1, 2026-07-28.

ADR-001. Rules are data, not code

Context. Every company has its own governance rules, and regulations change. Encoding them in Python would mean a release for every policy change. **Options.** Rules in code; rules in a database; rules as files loaded at call time. **Decision.** Packs as files, loaded at call time and anchored to the code rather than the working directory, with each rule carrying a machine-readable applicability spec. **Consequences.** Swapping the active rulebook is configuration, and because matching is deterministic the whole estate can be re-scored **without a single model call**. That is both the cheapest and the most convincing demonstration in the project. The limit is that regulatory tiers still need the model, so they do not move in a re-score, which is stated openly rather than hidden.

ADR-002. Never auto-block

Context. A governance tool that can switch off a production system is a tool nobody will install. **Decision.** The system flags and routes to a person. There is no blocking endpoint and no blocking code path. **Consequences.** The product is adoptable, and the promise is verifiable by absence rather than by policy. The cost is that governance depends on humans acting on flags.

ADR-003. Five inspectors, chosen per asset by an orchestrator

Context. Not every dimension applies to every asset. Running five reasoning agents on an asset that needs two is money spent for nothing. **Options.** Always run all five; a fixed rule table; an orchestrator agent that decides. **Decision.** An orchestrator agent picks the applicable inspectors, defaulting to all five when uncertain. **Consequences.** Cheaper runs on simple assets, and the decision is recorded in the state so it can be reviewed. The cost is one more model call and one more thing that can be wrong.

ADR-004. The roll-up, the matching and the decision are plain code

Context. If a model wrote the risk score, no benchmark number would mean anything, because the same asset could score differently twice. **Decision.** Risk roll-up, pack applicability, and the compliant-or-flagged decision are deterministic code. Agents never write them. **Consequences.** The measurements are reproducible and the pack swap is instant. The judgement stays with the agents; the arithmetic does not.

ADR-005. Every finding pins to a control id

Context. A governance finding that cannot name the rule it came from is an opinion. **Decision.** Each finding carries the policy rule or framework control it pins to, alongside evidence and a remediation, and is discarded if incomplete. **Consequences.** The audit trail is defensible, and a reviewer can trace any flag back to a rule.

ADR-006. Seeded assets carry empty audit chains

Context. A seeded estate with fabricated audit chains would make the tamper-evidence demonstration meaningless. **Decision.** The seeder writes no chain entries. A chain exists only where a real assessment ran. **Consequences.** Anything you see in an audit panel was genuinely produced by the pipeline, which is what makes the forged-entry demonstration land.

ADR-007. The benchmark answer key does not overlap the estate

Context. If the labelled assets were also in the seeded estate, an inspector could retrieve the answer through precedent instead of judging. **Decision.** Fifteen labelled assets, zero overlap with the 185-asset estate. **Consequences.** Tier accuracy and violation recall measure judgement rather than retrieval.

ADR-008. One model lane, no cloud

Context. An inventory of a company's AI estate is commercially sensitive in itself. **Decision.** One self-hosted open-weight model, no cloud client, no tier switch, inherited from the sibling contract-review system. **Consequences.** The privacy claim is provable by reading the code. The cost is model quality and per-wake GPU spend.

ADR-009. Pin the lane to one container and serialise calls

Context. Five inspectors fanning out made the platform start a second billed GPU whose model was still loading, which killed runs mid-way in the sibling system. **Decision.** Port the proven configuration: one container, a client-side lock, one retry on a stray server error. **Consequences.** Predictable cost, no mid-run container swaps. This repository ran an older router for a while and had to have the fix ported across, which is the clearest example of the cost of copy-forking. **Amended 2026-07-28, reverted 2026-07-29.** The lane briefly moved to vLLM with an AWQ checkpoint, batching up to 8 requests inside its one container, and the client-side lock was removed from the router here and in the sibling. It was rolled back the next day. The sibling benchmarked the swap over 13 contracts: 3.0x faster, but detection recall fell from 87.5% to 67.5% and the system produced 109 findings down to 69. Speed is not worth that in either product, so both routers went back to the lock and the lane back to bitsandbytes. The single-container pin was never the thing in question: it is the platform-side cap, not the lock, that prevents a second billed GPU.

One repair from that episode was **kept**, because it is a genuine bug fix rather than part of the swap: `_parse` now falls back to a tolerant read when the model emits Python literals (`True`) inside otherwise

valid JSON. Strict JSON rejected the whole reply, so an inspector's entire answer could be discarded over one capital letter, and `temperature=0` made the retry reproduce it exactly. See the sibling's ADR-014.

ADR-010. Take the first complete JSON object from model output

Context. Slicing from the first brace to the last broke deterministically whenever the model emitted a second object. One inspector failed both attempts at the identical character, twice. **Decision.** Decode the first complete object with a streaming decoder. **Consequences.** The failure class disappeared. The identical bug existed in the sibling system and was fixed there in the same session.

ADR-011. Fork the existing skeleton again

Context. This was the third system in the family, with days rather than weeks available. **Decision.** Fork the contract-review system, which had already forked the ticket system, and adapt. **Consequences.** The system was code-complete in a day. The cost is a third copy of several modules, and two bugs have already had to be fixed twice. The plan to replace copies with a shared package is recorded in the planning repository.

ADR-012. Remediation state lives inside the existing JSONB, not a findings table

Context. The remediation workflow (2026-07-28) needed owner, due date and status on every finding. The estate has one table, `assets`, with findings inside a JSONB array, and the open-finding counter and the decision endpoint already work directly on that array. **Options.** Normalise findings into their own table; extend the JSONB shape in place. **Decision.** Extend the JSONB shape. Four fields were added to the finding; existing findings read as unassigned and open, so no migration and no backfill ran. **Consequences.** A five-hour backend job instead of twelve, and every existing reader kept working. The accepted cost: a finding update is a read-modify-write of one asset's JSON document, so two concurrent edits to findings on the same asset can clobber each other. Fine for two reviewers on a demo estate; documented rather than solved.

ADR-013. The remediation screen is a status board, and dismissal is not a column

Context. Remediation work needed to be visible as work. The reference product class (VerifyWise and its peers) organises this as a board. **Options.** Dense rows with inline status edits; a four-column status board with drag and drop; both. **Decision.** A board with real drag and drop (dnd-kit) for the filtered views, chosen by the CEO from browser mockups, with the ALL view rendering as dense rows because 200 findings do not fit in columns. Two guards are requirements: `dismissed` is not a column and cannot be reached by dragging (dismissal stays on the override path, which requires a written reason), and a dismissed finding

cannot be revived by a drag (409). **Consequences.** The workflow reads as a product rather than an admin table, at roughly four extra hours of build cost. Drag listeners sit on a grip rather than the whole card, so the date picker inside a card does not start a drag. Every move writes an audit entry, keeping the tamper-evident chain the single story.

ADR-014. Evidence files get their own table, and cannot be deleted

Context. `awaiting_evidence` had been a board column since 2026-07-28 with nothing behind it: there was no way to attach the proof the column was waiting for. ADR-012 had just established the opposite instinct, that finding state belongs in the existing JSONB.

Options. Bytes in the asset JSONB, consistent with ADR-012; a separate `evidence` table; an object store or a directory on disk with a path in the database.

Decision. A separate table, with only the file's metadata mirrored onto the finding. ADR-012's reasoning does not carry over: it traded a migration for small scalars that every reader wanted anyway, whereas the asset blob is read in full on every estate view and file bytes inside it would tax every one of those reads to serve a column almost nobody opens. An object store was rejected as infrastructure this project does not otherwise need, and a disk path was rejected because the demo runs on two different laptops and a database row travels with the dump.

Consequences. One new table and one index, created through the same `CREATE TABLE IF NOT EXISTS` seam, so an existing database picks it up on the next boot with no migration step. The mirror can in principle drift from the table; the table is declared the source of truth for content and the mirror a display convenience, which is enough because nothing rewrites either after the upload. **There is no delete route, deliberately:** evidence is an audit record, and silent removal is the exact operation the hash chain exists to make impossible. Withdrawing a file would have to be a new recorded event, not an erasure.

07. Configuration and Secrets

Version 1, 2026-07-28.

1. Rules

- Real values live only in a git-ignored `.env`. The repository carries `.env.example` with blank placeholders.
- The lane URL and token are read when the model router is imported, so the API refuses to start without them.
- There is **no cloud model key**, by design.

2. Variables

Name	Required	Meaning
<code>DATABASE_URL</code>	yes	Postgres connection string, port 5435 , user and database governance . Use <code>127.0.0.1</code> to force IPv4
<code>AUTH_SECRET</code>	yes	Signs the session cookie
<code>PRIVATE_LANE_URL</code>	yes	HTTPS endpoint of the self-hosted model
<code>PRIVATE_LANE_TOKEN</code>	yes	Shared secret for that endpoint
Active policy pack	no	Which policy pack is live; switched through the API rather than by hand
Active framework pack	no	Which regulatory framework pack is live
<code>BRAND_NAME</code> , <code>BRAND_TAGLINE</code>	no	Branding
<code>CORS_ORIGINS</code>	no	Comma-separated origins the browser may call the API from, exact origin and no trailing slash. Defaults to <code>http://localhost:3000,http://localhost:3001</code>
<code>COOKIE_SECURE</code> , <code>COOKIE_SAMESITE</code>	no	Default to <code>false</code> and <code>lax</code> , which is what local development wants. Deployed, with the front end and the API on different domains, they must be <code>true</code> and <code>none</code> or the session cookie is silently dropped and login appears to do nothing
<code>FASTEMBED_CACHE_PATH</code>	no	Pinned inside the repository, after a temporary-directory embedding cache corrupted itself

The lane variables are shared with the sibling **contract-review system**, which points at the same single deployment. A blank pair here is a known and confusing failure: an assessment dies immediately with an invalid-address error, before the GPU is ever reached, so it costs nothing but looks alarming.

3. Ports

Service	Address
Control tower	http://localhost:3000
API	http://localhost:8000 , docs at <code>/docs</code>
Weaviate	http://localhost:8082 , gRPC 50053
Postgres	localhost: 5435

The data-store ports differ from both sibling systems so all three stacks can run at once. 5435 rather than the originally planned 5434, because another container already occupied that port.

4. Where secrets live

Secret	Home
Lane token	The <code>.env</code> on each machine, and a platform secret on the GPU side
Database password	The <code>.env</code> , and it must match the compose file
Session secret	The <code>.env</code>

The database password in the compose file is a long random string, not a guessable one. A common failure on a fresh machine is an environment file guessing the project's own name as the password; the two must be made to agree.

5. Common configuration mistakes

Symptom	Cause
The API will not start	A lane variable or the session secret is missing
Every assessment fails instantly with an invalid address	The lane variables exist but are blank
Database authentication failed	The environment file and the compose file disagree
Precedent search returns errors	The proxy is intercepting the vector database's gRPC port, or the collection was never seeded
The web app will not launch	Dependencies installed without their launcher shims; installing again creates them

6. Spend control

Every assessment wakes a rented GPU. A hard platform cap is set, the lane is capped at one container, and the benchmark has a single-asset switch used as a cost fence before any full run. The pack-swap demonstration is deliberately free.

08. Runbook

Version 1, 2026-07-28. Installation is in the repository `README.md` ; this is for running, recovering and diagnosing.

1. Daily start

```
docker compose up -d          # Postgres 5435 + Weaviate 8082/50053
uv run uvicorn api:app --reload # API on :8000
cd frontend && npm run dev      # control tower on :3000
```

One app at a time. All three sibling systems serve their API on :8000 and their front end on :3000, and every front end hardcodes `http://localhost:8000` . Stop this app before starting a sibling, or the tower will silently talk to the wrong API. The database ports are distinct, so the containers can all stay up.

Behind a TLS-intercepting proxy, set the exclusions in the same shell as the backend and any seed script:

```
export NO_PROXY=127.0.0.1,localhost no_grpc_proxy=127.0.0.1,localhost
```

2. Seeding

Command	Effect
<code>uv run python seed_users.py</code>	The administrator and reviewer accounts. Optional: on a system with no accounts the sign-in screen offers a one-time setup form that creates the founding administrator instead
The estate seed	185 synthetic assets, with deliberately empty audit chains
The precedent seed	Fills the precedent collection

All three are needed on a fresh machine and after any `docker compose down -v` . Embeddings are computed locally, so seeding costs nothing.

Rotating the seeded credentials

The seeded passwords are development defaults and **this repository is public**, so anybody can read them. They are fine on a laptop and unacceptable on anything reachable from the internet.

To retire them, supply an override and re-run the same seeder against the deployed database:

```
SEED_ADMIN_PASSWORD=... SEED_REVIEWER_PASSWORD=... DATABASE_URL=<the deployed database> uv
```

Re-running with an override **rotates** an account that already exists: it replaces the stored hash and keeps the role. Accounts are matched by address, so nothing is duplicated and nothing else is disturbed. Run it with no overrides and it prints a reminder that the published defaults are in use.

3. Health checks

Check	How	Healthy answer
API alive	GET /health	ok
Dependencies alive	GET /healthz	postgres and weaviate both ok, status ok. degraded names which one is down
Estate loaded	The tower home	185 asset cards
Sign-in	A seeded account	Reaches the tower
Model lane warm	A short probe	An answer in about a minute from cold

4. Routine operations

Task	How
Register an asset	Paste a paragraph on the tower; the card narrates its stages
Swap the rulebook	The packs screen, or the activate endpoint. Free, deterministic, instant
Sweep the estate	The sweep endpoint, over a slice
Route a flag	The flags screen. Never blocks anything
Read the executive brief	The brief screen

5. Incidents

Every assessment fails immediately with an invalid address

The lane variables are blank in this repository's environment file. This is the single most common failure here, because the lane really lives in the sibling repository's configuration and both projects share one deployment. It costs nothing, because the run dies before the GPU.

An assessment hangs, then times out

The lane is cold, or a container swap happened mid-run. Each node has a bounded wall clock and one retry. Warm the lane rather than retrying cold, because every cold wake costs money.

One inspector reports failed, the rest are fine

Expected: the assessment continues and the failure is visible. If it fails at the identical character on both attempts, that is truncated output rather than randomness.

The estate counts look frozen after a sweep or a re-score

This was a real bug: open findings were counted from the raw inspector output rather than from the canonical assessment, so counts silently stopped moving. It is fixed; if counts ever look stale again, that is

the first place to look.

Database authentication failed

The environment file and the compose file disagree on the password. The compose password is a long random string, not the project name.

The control tower will not start

Dependencies were installed but the launcher shims were not created. Install again.

Precedent search returns an error observation

The collection was never seeded on this machine, or the proxy is intercepting its port.

6. Before a demo

Follow `demo-script.md`, four beats:

1. **Register a live asset**, so the audience sees a real assessment run and the stages narrate. Warm the lane ten minutes ahead.
2. **Open the audit trail** and show the chain, then the forged-entry detection.
3. **Swap the rulebook** and watch the estate re-score. This is free and instant, and it is the strongest beat.
4. **Show the executive brief.**

Keep a previously registered live asset in the estate as a fallback so beat one never depends on the network.

7. Cost discipline

Assessments cost GPU money; the pack swap, the estate views, the brief and the audit trail do not. Rehearse on the free parts.

09. Test Plan and Report

Version 1, 2026-07-28.

1. Strategy

Layer	Cost	What it proves
Automated tests (tests/)	free, zero model calls	The deterministic seams: validation, fan-in, the audit chain, pack swapping, provenance, the reasoning loop, benchmark scoring
Live probes	free or cheap	That a registration really flows through the pipeline and leaves an intact chain
Benchmark (bench.py)	GPU money	Tier accuracy, violation recall, flag precision and audit completeness against a labelled answer key. See 10-benchmark-report.md

2. Running them

```
uv run python -m pytest tests/ -q
```

Use `python -m pytest`, not `pytest` alone, or collection cannot find the application package.

3. Coverage

80 tests, green in about a second, no model calls.

File	Tests	Covers
test_evidence.py	11	Evidence on a finding, driven through the real endpoints with both the asset store and the evidence table swapped for dicts: an upload lands in the table and mirrors its metadata onto the finding, a second file appends rather than replacing, the upload joins the asset's hash chain and the chain still verifies, an unsupported type and an empty file are refused with 400, a file over the ceiling with 413, a dismissed finding refuses evidence with 409, an unknown finding 404s, listing returns metadata with no bytes, a download returns the exact bytes with an attachment header, and a missing evidence id 404s
test_remediation.py	19	The remediation queue and board, driven through the real endpoints with the database swapped for a dict: the status vocabulary is enforced (a word outside the four board columns is refused with 422, and the error points at the override path), an unknown finding 404s, each filter (mine , unassigned , overdue , team , status) returns only matching findings, owner and due date set and clear correctly, a malformed date is refused, every change grows that asset's hash chain by exactly one and leaves the verifier intact, a dismissed finding refuses to be moved with 409, and an approved finding is still open and therefore still on the board

File	Tests	Covers
<code>test_flag_decision.py</code>	9	The reviewer's verdict on a flag, driven through the real endpoint with the database swapped for a dict: approve confirms and leaves the work open, override dismisses and keeps the reason, an override with no reason is refused, an unknown verdict is refused, a flag cannot be decided twice, unknown finding and unknown asset both 404, the verdict joins the hash chain and the chain still verifies, the decision is actually persisted, and the owner of a finding's remediation is refused from deciding it (maker-checker, 403)
<code>test_audit.py</code>	8	The hash chain and its verifier, including the newer hashed timestamp and actor fields: a backdated timestamp or a swapped actor breaks the chain, and entries written before those fields existed still verify
<code>test_tier_override.py</code>	5	The human correction of the AI-assigned risk tier: it changes the tier, records who and why, joins the hash chain intact, refuses a missing reason and an unknown tier, conflicts on a no-op, and 404s on a missing asset
<code>test_exports.py</code>	4	The CSV exports: register and findings carry headers and quoted rows, the audit CSV marks intact entries per row, and a missing asset 404s
<code>test_sweep_async.py</code>	3	The backgrounded sweep: start returns immediately, status reports done with the report, an error is visible rather than silent, and a second start while running conflicts
<code>test_bench_scoring.py</code>	5	The benchmark's own scoring, so the numbers it reports are trustworthy
<code>test_fanin.py</code>	5	Finding validation, the casing fix on the tier, and the deterministic roll-up
<code>test_state.py</code>	4	Shapes and the risk roll-up
<code>test_provenance.py</code>	3	That seeded assets and live registrations stay distinguishable
<code>test_react.py</code>	3	The reasoning loop
<code>test_config_swap.py</code>	1	That activating a different pack re-scores deterministically

Testing the benchmark's scoring is worth calling out: a measurement harness nobody checks is a good way to publish a wrong number confidently.

4. What testing and live probing have caught

- **Open-finding counts were computed from raw inspector output rather than the canonical assessment**, so estate counts would have silently frozen after any sweep or re-score. Found by testing, fixed the same session.
- **The reasoning loop's JSON parser broke deterministically** when the model emitted a second object; one inspector failed twice at the identical character. Fixed here and in the sibling system.
- **This repository was running an older model router** than its sibling, missing the single-lane lock and the retry. Ported across.
- **The tamper-evidence claim was verified end to end** by forging an entry directly in Postgres and confirming the endpoint reported the break at the exact index.

5. Live verification of the flag decision, 2026-07-28

Run against the real API and the real estate database, no model calls: signed in as an administrator, took an undecided open finding from a seeded asset, confirmed that an override with an empty reason is refused with 422, recorded an override with a reason, confirmed a second decision on the same flag returns 409, and read the audit endpoint back. The chain grew by one entry naming the finding, the verdict, the reviewer and the reason, and still verified as intact. The estate row's open-finding count dropped by one. The probe's write was then reverted, so the seeded estate is untouched.

6. Live registration verification

One asset was registered through the tower on a warm lane and assessed end to end: the correct regulatory tier for an employment-screening system, five findings across four inspectors covering a missing bias test, no named human reviewer and no kill switch, an intact eight-entry audit chain, in about 220 seconds. It is retained in the estate marked as a live registration, which is how it stays distinguishable from the seeded 185.

7. Known gaps

Gap	Why it matters
Thin endpoint-level coverage	The flag-decision and remediation endpoints are covered by tests. Registration, packs, sweep and the brief are still verified by hand and by live probes. This became possible on 2026-07-28 : the module used to create its database tables at import, so it could not be imported without Postgres, which is why no endpoint tests existed. Table creation moved into the startup hook
No frontend test of the drag interaction	The board's drag and drop is verified by hand; both the board and the ALL rows view read the same endpoint, so a card can always be moved without dragging if it fails live
No test asserting asset text never leaves for a third party	The claim rests on the absence of a cloud client
No frontend tests	The tower is verified by hand
Tier accuracy is only measurable through the paid benchmark	There is no free proxy for it

8. Test data

A 185-asset synthetic estate, and a **15-asset labelled answer key with zero overlap** with it, so the benchmark measures judgement rather than retrieval. Seeded assets carry empty audit chains, so any chain in the system was produced by a real run.

10. Benchmark Report

Version 1, 2026-07-28. Raw results are committed as `bench_governance.json`.

1. Method

Fifteen labelled assets, each with a known correct regulatory tier and a known set of violations, and **zero overlap with the 185-asset estate**, so nothing can be retrieved instead of judged. Each asset runs through the full pipeline on the live model lane and is scored on four measures:

Measure	Definition
Tier accuracy	Assets given the correct regulatory risk tier
Violation recall	Known violations the system found
Flag precision	Of the flags raised, the share that were in the answer key
Audit completeness	Assets whose audit chain covers every step, intact

2. Results

Measure	Result
Assets	15
Completed	15 of 15, zero retries, zero errors
Tier accuracy	73.3%
Violation recall	74.3%
Flag precision	46%
Audit completeness	1.0

A single-asset probe before the full run completed cleanly with the correct tier and a complete audit chain, which is the cost fence that protects the full run.

3. What these numbers mean

- **Every asset completed on the first attempt.** For a five-agent parallel pipeline on one rented GPU, zero errors and zero retries across fifteen assets is the reliability result, and it is the number that took the most engineering to earn.
- **Audit completeness is exactly 1.0**, which is the point of the product: whatever the model judged, the record of how it judged is complete and verifiable.
- **Tier accuracy of 73 percent is the honest headline, and it is the weakest link.** One asset was rated high where the answer key says unacceptable. Under-rating is the dangerous direction of error for a governance tool, and it should be said plainly rather than averaged into a friendlier number.

- **Violation recall of 74 percent** means the system finds about three in four known problems. Useful as a first pass, not a substitute for review.
- **Flag precision of 46 percent** means roughly half of the flags were not in the answer key. Some are genuine issues the corpus author did not label, some are noise. They cannot be separated without a second reviewer, so the raw figure stands.

4. Limits, stated plainly

- Fifteen assets is a batch, not a statistical sample.
- The corpus and its answer key were written by the same person who built the system, which is the strongest bias here.
- Precision is measured against a key that was never intended to be exhaustive, so it understates real precision by an unknown amount.
- The measures depend on a warm lane; a cold start adds about a minute per asset.
- Nothing measures whether a remediation is good advice.

5. What the numbers are not

They are not a claim that the system can replace a governance function. They are evidence that a multi-agent pipeline can produce a **complete, control-referenced, tamper-evident record** for an estate, with roughly three-quarters accuracy on judgement, which is exactly how it should be described to a reviewer.

6. Reproducing

```
uv run python bench.py --only AI-9001    # one asset, the cost fence
uv run python bench.py                   # the full labelled set, one warm window
```

The benchmark's own scoring is covered by unit tests, so the numbers it prints are checked as well as the system it measures.

11. Non-Functional Requirements

Version 1, 2026-07-28.

1. Performance

Property	Target	Measured
One asset assessed	minutes, unattended	about 220 seconds on a warm lane, live
First call after the lane idles	under two minutes	roughly 60 to 90 seconds
Per-node ceiling	bounded, with one retry	1200 seconds
Registration response	immediate	The asset is parked as processing before any model runs
Estate re-score after a pack swap	instant, and free	Deterministic matching, no model calls
Full labelled benchmark	one warm window	15 assets, zero retries

The re-score deserves emphasis: changing the rulebook for 185 assets costs nothing and finishes immediately, because applicability is plain code.

2. Scale

Single concurrent user, an estate of 185 synthetic assets, one assessment at a time. The estate views, the sweep and the brief operate over the whole estate; only the per-asset pipeline is expensive.

3. Availability

No availability target, no failover. The guarantee that is met: **a failure is recorded rather than hidden**. A failing inspector is marked failed with a note and the assessment continues; a failing node records into the state instead of raising; an asset that cannot be assessed ends in an explicit error status rather than staying at processing.

4. Privacy

Requirement	Status
Asset descriptions never reach a third-party model	Met: one lane, no cloud client in the codebase
Embeddings computed locally	Met, with the cache pinned inside the repository
No open signup	Met

An inventory of a company's AI estate is commercially sensitive in itself, which is why this system inherited the single-lane design rather than the routing grid of the original ticket system.

5. Auditability

This is the system's strongest property and the one it is really selling.

- Every step of every assessment appends to a hash chain stored with the asset.
- The chain is verified when read and reports the first broken index.
- Tested against an entry forged directly in the database: the break was reported at the exact index.
- Seeded assets carry **no** chain entries, so any chain in the system was produced by a real run.
- Measured audit completeness across the labelled benchmark: **1.0**.

6. Determinism

Three things must never become model calls, because the benchmark and the pack swap both depend on them: the risk roll-up, pack applicability matching, and the compliant-or-flagged decision. All three are plain code today and are covered by tests.

7. Cost

The GPU lane bills per warm window. Controls: a hard platform cap, a single-container lane, a single-asset switch as a cost fence before any full run, and a demonstration path whose strongest beat costs nothing.

8. Maintainability

This system is a fork of a fork. Several modules are near-identical to their siblings, and this repository has already been caught running an older copy of the model router than its sibling. The duplication is measured and the shared-package decision is recorded in the planning repository; until it happens, a fix in a shared-looking module must be checked in all three repositories.

12. Security Review

Version 1, 2026-07-28. A threat model for a single-machine demonstration system that holds a map of a company's AI estate.

1. Assets

The inventory itself, which is commercially sensitive: what AI the company runs, who owns it, what data it touches, and where it is weak. The findings, which are a list of the company's governance failures. Account credentials, the model lane token, and the audit chains.

A list of every unpatched governance gap in an organisation is a target in its own right, which is worth saying out loud.

2. Trust boundaries

Boundary	Other side
Browser to API	Anyone who can reach the port
API to the model lane	An internet-reachable GPU endpoint
API to data stores	Local Docker containers
Packs on disk	Whoever can write to the repository

3. Threats and controls

#	Threat	Control today	Residual risk
1	An outsider creates an account and reads the estate	No open signup ; an administrator creates every account	The administrator account is a single point of trust
1a	An outsider claims the founding administrator through the first-run setup route	POST /auth/bootstrap is unauthenticated by necessity, but it counts the accounts first and refuses with 403 the moment any exists. On a seeded or running system it is already closed	A system that is deployed but never seeded is claimable by the first visitor. Whoever installs it must complete setup before publishing the URL. The same applies to the sibling contract-review system
2	Password guessing	Hashed passwords	No rate limiting or lockout
3	Session theft	Signed cookie with a secret from the environment	Not marked secure, because the demonstration runs over plain HTTP

#	Threat	Control today	Residual risk
4	A reviewer quietly weakens the rules by editing a pack	Packs are files on disk; the active pack is recorded and every finding pins to a control id	Nothing signs or version-controls the packs at runtime. This is the most product-specific weakness: the rulebook is the trust anchor and it is a writable file
5	Someone edits the record to hide a finding	Hash chain per asset, verified on read, tested against a forged database row	Tamper evident, not tamper proof
6	Asset descriptions leak to a model provider	One self-hosted lane, no cloud client anywhere	Text does travel to a rented endpoint over an authenticated connection
7	The lane token is stolen and the GPU budget spent	Shared secret and a hard platform cap	The endpoint is internet reachable
8	Prompt injection inside a registration description, to suppress a finding or claim a lower tier	Findings must carry evidence and a control id, the roll-up and the decision are plain code, and a person reviews flags	Not systematically tested. A registration is free text written by the asset's own owner, who has an incentive to look compliant, so this is a realistic attack rather than a theoretical one
9	The system is used to switch something off	There is no blocking endpoint and no such code path	None
10	Every reviewer sees the whole estate	Two roles only	No per-team or per-business-unit boundary
11	Secrets committed	Only blank placeholders are committed	Nothing scans commits

4. What would have to change before real use

1. Signed or version-controlled packs, with a record of who activated which pack and when.
2. Per-team access boundaries on the estate.
3. Rate limiting and lockout, HTTPS, and a secure session cookie.
4. A tested position on injection inside self-reported registrations, given that the person registering an asset benefits from a clean result.
5. Encryption at rest for the estate database.

5. Deliberate non-goals

No penetration test, no dependency scanning, no enforcement capability of any kind. The absence of enforcement is a product decision, not an oversight, and it removes an entire class of risk: this system cannot take anything down.

13. Privacy and Data Handling

Version 1, 2026-07-28.

1. Position

The estate in this system is entirely synthetic. The design assumes real corporate data, because the inventory of a company's AI systems is sensitive even when no personal data is in it: it describes what the company runs, who owns it, what it touches, and where the gaps are.

2. What data the system holds

Data	Where
The registration description, as written	The asset row, inside the state
The canonical asset record: name, type, owner, lifecycle, purpose, data touched, who is affected, whether a human reviews it	The asset row
Findings, with evidence and remediation	The asset row
The risk tier, risk level, and the decision	Real columns plus the state
Assessment summaries	The precedent collection
Audit chains	Inside the state
Account details	The accounts table, passwords hashed

Note what is **not** here: no model weights, no training data, no production traffic, no personal data belonging to the people those AI systems affect. The system reasons about descriptions of AI assets, not about the assets themselves. That is a deliberate scope limit and it keeps the privacy surface small.

The one field that names a person is the asset owner, which is ordinary workplace directory information.

3. Where data flows

Flow	Destination
Inventory, orchestration, inspection, sweep	The self-hosted model endpoint on rented GPU infrastructure
Embeddings for precedent search	Computed locally; nothing is sent out to be indexed
Storage	Local Postgres and local Weaviate
Pack matching and re-scoring	Entirely local, no model involved

There is no third-party model provider anywhere in the system. The honest qualification is the same as its siblings: the lane is self-hosted but not on premises, so descriptions do travel to that endpoint over an

authenticated connection.

4. Retention

Nothing is deleted. Assets, assessments, findings and audit chains are kept indefinitely, which is appropriate for a governance record: the value of an audit trail is that it does not disappear. There is no purge and no deletion endpoint.

For real use this would need a retention policy that distinguishes the **audit chain**, which should be immutable and long-lived, from the **asset description**, which may need correcting or removing.

5. Access

Two roles: reviewer and administrator. Every reviewer sees the whole estate. There is no per-team boundary, which is listed in [12-security-review.md](#) as a control to add before real use, since a governance estate is exactly the kind of data a company would want compartmented.

6. Precedent and confidentiality

Finished assessments are filed into a shared precedent collection and retrieved for later assets. Within one company that is the intended learning loop and raises no confidentiality problem, unlike the sibling contract-review system where precedent crosses client boundaries. If this were ever run as a multi-tenant service, the same boundary work would be needed here.

7. Gaps to close before real data

1. A retention policy separating the immutable chain from the mutable description.
2. Per-team access boundaries.
3. Encryption at rest.
4. A record of which model version assessed which asset, alongside the chain.
5. A correction path: today an incorrect asset record can only be superseded, not amended with a recorded reason.

From the fifteen-asset labelled benchmark: every asset completed on the first attempt with zero errors, tier accuracy 73.3 percent, violation recall 74.3 percent, flag precision 46 percent, and audit completeness 1.0. Detail and caveats in [10-benchmark-report.md](#).

Live: one registration produced the correct tier for an employment-screening system and five findings across four inspectors, in about 220 seconds.

4. Known failure modes

Failure	How it shows	What contains it
Under-rating a tier	An asset rated high where the answer key says unacceptable	Nothing automatic. This is the most consequential error the system makes and it is reported openly rather than smoothed over
Over-flagging	Roughly half of flags were not in the answer key	Every finding carries evidence and a control id, so a reviewer dismisses a wrong one quickly
Second JSON object after the answer	A deterministic parse failure, twice at the identical character	The parser takes the first complete object
Truncated output	Same fingerprint	A generous token ceiling per call
Inconsistent capitalisation of the tier	Downstream comparisons silently fail	Lower-cased once, at fan-in
Cold container	First call after idle takes about a minute	Warm the lane before a demonstration
Self-serving registrations	An owner describes their system in flattering terms and gets a clean result	Not solved. Findings rest on the description given, and the system has no way to inspect a running model

That last one is the honest boundary of the whole product: it governs **what it is told**, not what is actually deployed.

5. What is not measured

Whether a remediation is good advice. Whether a flag a reviewer dismissed was in fact real. How the model performs on assets unlike the synthetic estate, which was written by the same person who built the system.

6. Appropriate and inappropriate use

Appropriate: a first-pass inventory and assessment of AI assets, producing a control-referenced, tamper-evident record that a human governance function reviews.

Inappropriate: treating a clean result as assurance; using tiers for a regulatory filing without human confirmation, given 73 percent accuracy; any use where the description of an asset cannot be

independently checked.

7. Human oversight

Every flag is a work item for a person. The system has no ability to block, disable or quarantine anything, and there is no code path that could. The decision it makes is a label, not an action.

15. Risk Register

Version 1, 2026-07-28.

Open risks

#	Risk	Likelihood	Impact	Mitigation
R-1	Tier accuracy of 73 percent, with at least one asset under-rated , and under-rating is the dangerous direction for a governance tool	High	High	Reported openly in the benchmark and the model card. Every tier is a human's to confirm. The first quality work to resume if time allows
R-2	Flag precision of 46 percent buries reviewers in flags they must dismiss	Medium	Medium	Findings carry evidence and a control id, so each is quick to judge. Not separable from genuine discovery without a second reviewer
R-3	The rulebook is a writable file with nothing signing or versioning it at runtime	Medium	High	The active pack is recorded and every finding pins to a control id. Signing and change history are named as pre-production work
R-4	A live demonstration runs on a cold lane	Medium	High	Warm ten minutes ahead, keep a previously registered live asset in the estate, and lead with the free pack-swap beat
R-5	GPU spend exhausts the credit	Medium	Medium	Hard platform cap, one-container lane, single-asset cost fence, and a demonstration path whose strongest beat is free
R-6	An asset owner describes their system flatteringly and gets a clean result	High	Medium	Unsolved by design: the system governs what it is told. Stated as the product's boundary rather than hidden
R-7	Prompt injection inside a registration description	Medium	Medium	Evidence and control ids, deterministic roll-up and decision, human review of flags. Not systematically tested
R-8	Duplicated modules mean a fix in a sibling never lands here	Medium, already realised once	Medium	This repository was found running an older model router than its sibling. The duplication is measured and the shared-package plan is written
R-9	Every reviewer sees the whole estate	Low here, High in production	Medium	Named as a control to add before real use
R-10	Documentation drifts from a fast-moving codebase	Medium	Low	Every document is dated and states what it was checked against

Closed risks

#	Risk	How it closed
C-1	Estate open-finding counts were computed from raw inspector output rather than the canonical assessment, so counts would have silently frozen after any sweep or re-score	Found by testing, fixed the same session
C-2	A second JSON object in model output broke parsing deterministically, killing one inspector twice	The parser takes the first complete object; fixed here and in the sibling system
C-3	The model router here lacked the single-lane lock and the retry its sibling had gained	The proven version was ported across
C-4	The lane credentials in this repository were blank placeholders, so the first benchmark run died before reaching the GPU	Filled by hand; no money was wasted because the run died early
C-5	The tamper-evidence claim was untested	Verified by forging an entry directly in the database and confirming the endpoint reported the break at the exact index
C-6	The seeded estate could have made the audit demonstration meaningless	Seeded assets deliberately carry empty chains
C-7	The benchmark answer key could have leaked into the estate, so retrieval would substitute for judgement	Fifteen labelled assets with zero overlap
C-8	The interface was indistinguishable from the sibling contract-review system	Reskinned to a distinct dark theme by redefining a small set of design variables rather than touching components

16. Traceability Matrix

Version 1, 2026-07-28. Each requirement from [01-requirements.md](#), the code that satisfies it, and the test or measurement that proves it.

Functional

Req	Verdict	Code	Proof
FR-1 register from a plain-language description	MET	<code>POST /assets , inventory_agent in app/agents.py</code>	Live registration, 2026-07-27
FR-2 central inventory	MET	<code>app/store.py , GET /assets , the estate seed</code>	185 seeded assets
FR-3 policy and framework packs as data	MET	<code>app/packs.py , data/policy_packs , data/framework_packs</code>	<code>test_config_swap.py</code>
FR-4 five inspection dimensions	MET	Five inspector agents, parallel step in <code>app/graph.py</code>	Benchmark: violation recall 74.3%
FR-5 choose applicable inspectors	MET	<code>orchestrate_agent , conditional fan-out in app/graph.py</code>	Recorded in the state per asset
FR-6 regulatory risk tier	MET	<code>risk_assessment_agent , lower-cased at fan-in</code>	Benchmark: tier accuracy 73.3%; <code>test_fanin.py</code> covers the casing
FR-7 deterministic risk roll-up	MET	<code>risk_rollup() in app/state.py</code>	<code>test_state.py , test_fanin.py</code>
FR-8 findings complete and control-referenced	MET	<code>valid_finding() , _stamp in app/agents.py</code>	<code>test_fanin.py , test_state.py</code>
FR-9 route a flag, never auto-block	MET	<code>POST /flags/{id}/route ; no blocking endpoint exists</code>	Proved by absence: there is no such code path
FR-9a record the reviewer's verdict, with a reason for any dismissal	MET	<code>POST /flags/{id}/decision in api.py ; the verdict joins the asset's hash chain</code>	<code>test_flag_decision.py</code> (8 tests) plus a live probe against the real database

Req	Verdict	Code	Proof
FR-9b remediation as tracked work: owner, due date, status, board	MET	GET /remediation and PATCH /flags/{id} in api.py, store.list_findings(), the /remediation board with dnd-kit	test_remediation.py (19 tests): vocabulary enforced, filters exact, every change audit-chained, dismissed findings immovable, approved findings still open
FR-9c evidence attached to a finding, downloadable and audit- chained	MET	POST/GET /flags/{id}/evidence and GET /evidence/{id} in api.py, the evidence table and store.add_evidence/list_evidence/get_evidence, the card control on the /remediation board	test_evidence.py (11 tests): upload lands and mirrors, chain grows by one and still verifies, type/size/empty refused, dismissed findings refuse evidence, listing carries no bytes, download returns the exact bytes
FR-10 swap the rulebook and re-score	MET	POST /packs/activate, rescore_policy() in app/sweep.py	test_config_swap.py; measured 194 findings across 111 assets to 147 across 114, 86 re-scored, chains intact
FR-11 estate sweep	MET	run_sweep() in app/sweep.py, three sweep agents	Exercised live
FR-12 executive brief	MET	GET /brief	Exercised live
FR-13 tamper- evident audit per asset	MET	app/audit.py, GET /assets/{id}/audit	test_audit.py (5 tests); verified live against a forged database row; benchmark audit completeness 1.0
FR-14 administrator- created accounts	MET	app/users.py, the users endpoints	Role gates in the API

Non-functional

Req	Verdict	Evidence
NFR-1 rules as data, swappable without a restart	MET	Call-time pack loading anchored to the code; test_config_swap.py
NFR-2 no third-party model	MET	One lane in app/router.py; no cloud client exists. Not covered by a test

Req	Verdict	Evidence
NFR-3 unattended assessment in minutes	MET	About 220 seconds live; 15 of 15 completed in the benchmark
NFR-4 a stuck agent cannot hang an assessment	MET	<code>guarded()</code> in <code>app/graph.py</code>
NFR-5 tampering detectable	MET	<code>test_audit.py</code> plus the live forged-row check
NFR-6 deterministic parts stay deterministic	MET	Roll-up, matching and decision are plain code; <code>test_fanin.py</code> , <code>test_config_swap.py</code>
NFR-7 synthetic estate, secrets outside the repository	MET	185 synthetic assets; ignored environment file
NFR-8 reuse of the skeleton	MET	Forked from the contract-review system; duplication measured in the planning repository

Coverage summary

64 automated tests, no model calls, covering the audit chain, the fan-in reducer including the tier casing fix, validation, the reasoning loop, provenance (seeded against live), the pack swap, the benchmark's own scoring, the flag-decision endpoint, the remediation queue and board endpoints, and the evidence upload, listing and download routes.

The gaps, stated plainly: endpoint coverage is still **partial**, since only the flag-decision and remediation endpoints are exercised by tests (registration, packs, sweep and the brief are verified by hand and by live probes), and there is **no test asserting that asset descriptions can never reach a third party**, which currently rests on the absence of any cloud client in the codebase.

17. User Guide

Version 1, 2026-07-28. For the reviewer using the control tower, and the administrator running it.

1. Signing in

There is no self-service signup. An administrator creates your account. Sign in and you land on the tower.

The very first time the system is ever started, there are no accounts at all, so there is nobody to create yours. The sign-in screen notices this and turns itself into a one-time setup form: fill in an email address and a password, and it creates the founding administrator. That administrator then creates everyone else. Once a single account exists this screen never appears again, and the setup route refuses anyone who tries it later. On a development machine `seed_users.py` gets you to the same place.

2. The estate

The tower lists every AI asset the company has registered: what it is, who owns it, where it is in its lifecycle, its risk level, its regulatory tier, and how many findings are open against it.

An asset being assessed narrates its progress: intake, orchestrating, inspecting, rolling up, done.

3. Registering an asset

Paste a paragraph describing the system in plain language: what it does, who owns it, what data it touches, who it affects. You do not fill in a form; the system turns the paragraph into a structured record.

Then it assesses the asset across up to five dimensions:

Reviewer	Asks
Policy compliance	Does this break one of our own rules?
Risk assessment	What regulatory tier is this, and what makes it risky?
Data governance	What data does it use, on what basis, kept for how long?
Responsible AI	Has anyone tested it for bias, is it transparent, does a human oversee it?
Security and third party	What does it depend on, and who else can reach it?

Not all five run on every asset: an orchestrator decides which apply, and records that decision.

Expect a few minutes. If it is the first assessment after a quiet period, add a minute for the model server to wake.

4. Reading an assessment

Each finding tells you five things: which rule or control it pins to, how severe it is, what it means in plain words, the evidence in the record that triggered it, and what to do about it. A finding that arrives incomplete is discarded before you see it.

The asset ends up either **compliant** or **flagged**. Flagged means there is work for a person. It does not mean the asset has been stopped, and the system has no ability to stop anything.

5. Flags

Route a flag to whoever should deal with it. That is the whole enforcement model: a work item for a human being.

On each finding you can also record what you concluded: **approve** confirms it is real (it stays open, because the fixing still has to happen), **override** dismisses it and requires a written reason. A flag can only be decided once.

5a. The remediation board

The remediation screen shows every finding as a card on a four-column board: open, in progress, awaiting evidence, closed. Drag a card to move it, set an owner and a due date on the card, and filter to MINE, MY TEAM, UNASSIGNED or OVERDUE with the chips. The ALL view shows the whole estate as rows instead, which is the fallback if dragging ever misbehaves in a demo.

Two things the board will refuse, on purpose: you cannot drag a card to dismissed (dismissal is an override and needs a written reason), and a dismissed finding cannot be dragged back to life. Every move you make is written into that asset's audit chain.

Evidence on a card

The bottom of each card says either NO EVIDENCE or EVIDENCE with a count. Click it to open the list, where every file downloads by name, and use + **ATTACH PROOF** to add one: a screenshot of the corrected setting, a signed policy, whatever shows the work happened. Images, PDF, plain text and CSV up to 10 MB are accepted, and anything else is refused with the reason on screen.

The upload joins the asset's audit chain, so the proof and the claim live in the same record. **Files cannot be removed**, because evidence is an audit record and quietly deleting one is exactly what the chain exists to prevent. A dismissed finding takes no evidence.

6. Swapping the rulebook

This is the feature worth understanding. The rules are data, not code. Activating a different policy pack re-scores every asset in the estate against the new rules **immediately and at no cost**, because rule applicability is decided by plain matching rather than by the model.

One honest limit, worth knowing before anyone asks: the regulatory **tier** does not move in a re-score. Tiering needs the model, so it changes only when an asset is reassessed.

7. The audit trail

Every asset has a chain recording each step of its assessment, where each entry is sealed against the one before it. Open the audit panel to read it. If any past entry were altered, the panel reports exactly where the chain broke. Seeded demonstration assets have no chain at all, so anything you see was produced by a real run.

8. The sweep and the brief

The sweep runs three extra reviewers across a slice of the estate: one looks for monitoring and drift signals, one for what has changed in the regulations, and one produces reporting. The executive brief rolls the estate up for leadership.

9. For administrators

The people page creates reviewer and administrator accounts and deactivates them. Activating a pack is an administrator action, since it changes how the whole estate is judged.

10. Worth knowing

- **Nothing is ever auto-blocked.** By design, and there is no code that could.
- **Assets never leave for an outside model provider.** One self-hosted model, no cloud path.
- **The system governs what it is told.** It reads descriptions of AI systems, not the systems themselves, so a flattering registration produces a flattering result. Treat a clean assessment as a starting point, not assurance.
- **Tiers are about three-quarters accurate.** Confirm every one that matters.

18. Release Notes

Version 1, 2026-07-28. No version tags; 18 commits, from 2026-07-23 to 2026-07-27. The system was built code-complete in about a day and hardened over the following three.

2026-07-29, evidence

- **The `awaiting_evidence` column finally has something behind it.** A finding can now carry uploaded files: the screenshot of the corrected setting, the signed policy, whatever proves the work was done. Each card on the board opens to list its files and take a new one, and each file downloads by name.
- **The upload joins the hash chain**, like every other change to a finding, so the proof and the claim that the work happened sit in the same tamper-evident record.
- **Bytes went in a table, not the asset blob**, reversing the instinct from the remediation release for a stated reason (ADR-014): the blob is read in full on every estate view, and files inside it would tax every one of those reads. Only the metadata is mirrored onto the finding, which is what lets a card show its count without a second query.
- **There is no delete route, deliberately.** Evidence is an audit record; silent removal is the operation the chain exists to prevent.
- A dismissed finding refuses evidence (409), on the same reasoning that stops it being dragged. Eleven tests came with it, taking the suite from 53 to 64.

2026-07-28 later, remediation becomes somebody's job

- **The remediation board.** A finding now carries an owner, a due date and a status (`open` , `in_progress` , `awaiting_evidence` , `closed` , `dismissed`), stored inside the existing JSONB shape so no migration ran. A new `/remediation` screen shows the work as a four-column board with real drag and drop, filterable to mine, my team, unassigned or overdue, and the ALL view renders as dense rows because two hundred findings do not fit in columns.
- **Every change is in the chain.** Setting an owner, moving a card, changing a due date: each appends an entry to that asset's tamper-evident hash chain. Remediation state cannot change without a trace.
- **Dismissal stays a judgement.** `dismissed` is not a board column and cannot be reached by dragging; that path stays on the override verdict, which requires a written reason. A dismissed finding refuses to be revived by a drag (409). And an approved finding stays `open` , because approval is the start of remediation, not the end; the board is where approved findings land.
- The seeder now walks findings through a deterministic cycle of owners, due dates and statuses, so the board opens populated; it still writes no audit entries and never overwrites a dismissed finding.
- Nineteen endpoint tests came with it, taking the suite from 34 to 53. Two bugs were fixed on the way: the user seeder ran at import (so merely importing it needed a live database), and the demo script still named the abandoned side-by-side ports.

2026-07-28, the reviewer's verdict

- **Approve and override on flags.** Routing already said who should look at a flag; nothing recorded what they concluded, so a flag stayed open forever and the reviewer's judgement left no trace. `POST /flags/{id}/decision` records it: approve confirms the finding and leaves the remediation work open, override dismisses it and **requires a reason**, because a dismissal with no reason is exactly what an auditor objects to. Either verdict appends to the asset's hash chain, so the human decision sits inside the tamper-evident record rather than beside it. A flag can only be decided once.
- This matters at the measured flag precision of 46 percent: a reviewer will dismiss roughly half of what the agents raise, and until now there was nowhere to say why.
- The control tower shows the two buttons on each finding, with an inline reason box for an override and the recorded verdict afterwards.
- **Enabling fix:** the module created its database tables at import, so it could not be imported without a live Postgres, which is why this repository had no endpoint tests at all. Table creation moved into the startup hook, the same fix the ticket-triage system needed. Eight endpoint tests came with it, taking the suite from 26 to 34.

2026-07-27, the tower gets its own face

- **Dark soft-elevation reskin.** The interface was previously indistinguishable from the sibling contract-review system, because both forked the same styling. The reskin was done by redefining a small set of design variables rather than by touching component classes, so no component changed.
- **Executive dashboard metrics** on the tower.
- Login branding and the post-login redirect corrected: both still pointed at the sibling system, which was the cause of a post-login page-not-found.
- Ports aligned with the sibling systems for the application, while keeping distinct database ports so all three stacks can run at once.
- **A full README**, which the project had been missing.
- **First live registration through the tower**, an employment-screening system: correct regulatory tier, five findings across four reviewers covering a missing bias test, no named human reviewer and no kill switch, an intact eight-entry audit chain, in about 220 seconds. Retained in the estate marked as a live registration.

2026-07-26, measurement

- **Full fifteen-asset benchmark committed:** all fifteen completed with zero retries and zero errors, tier accuracy 73 percent, violation recall 74 percent, flag precision 46 percent, audit completeness 1.0.

2026-07-25, correctness fixes shared with the sibling

- **The reasoning loop now takes the first complete JSON object** from model output. The previous parse broke deterministically whenever the model emitted a second object, and it killed the responsible-AI reviewer on both attempts at the identical character.
- **The model router gained the single-lane lock and the retry** it had been missing. This repository had been running an older copy than its sibling, which is the clearest illustration of what copy-forking

costs.

- The lane credentials in this repository were blank placeholders; a benchmark run died before reaching the GPU, at no cost.

2026-07-24, everything else

- The nightly sweep with three additional reviewers, on-demand flag routing, and the executive brief.
- The control tower interface.
- **Live pack swap with estate-wide re-scoring**, deterministic and free: one pack produced 194 findings across 111 assets, the other 147 across 114, with 86 assets re-scored and every audit chain intact.
- The free test layer and the benchmark harness, including tests of the benchmark's own scoring.
- The demonstration script, four beats.
- **Real bug found by testing**: open-finding counts were computed from raw inspector output rather than from the canonical assessment, so estate counts would have silently frozen after any sweep or re-score.

2026-07-23, the pipeline

- The per-asset graph: inventory, orchestrator, a conditional fan-out to five inspectors, a plain-code fan-in, and a deterministic decision.
- The registration endpoint, which is the demonstration's hero path.
- Packs as data, with an anchored call-time loader, so a rulebook swap needs no restart.
- The 185-asset synthetic estate and a 15-asset labelled answer key with zero overlap, with seeded assets deliberately carrying empty audit chains.
- Seven read-only tools.

Known issues carried forward

- Tier accuracy is 73 percent and at least one asset is under-rated, which is the dangerous direction of error.
- Flag precision is 46 percent against a key that was never meant to be exhaustive.
- A pack swap re-scores policy findings but not regulatory tiers, which need the model.
- The rulebook is a writable file with nothing signing or versioning it at runtime.
- Endpoint tests cover the flag-decision and remediation routes; registration, packs, sweep and the brief are still verified by hand.
- A finding update is a read-modify-write of one asset's JSON document, so concurrent edits to the same asset can clobber each other. Accepted for a two-reviewer demo estate.
- Evidence file upload (`evidence_files`) is a declared field with no upload endpoint yet.

19. Handover

Version 1, 2026-07-28.

1. Get it running

Repository `README.md` for installation, [08-runbook.md](#) for daily operation. Docker, `uv`, Node, and an environment file. Note the database port is **5435**, not the sibling systems' ports, so all three database stacks can run at once. The applications themselves cannot: **all three systems run their API on :8000 and their front end on :3000, and every front end hardcodes `http://localhost:8000`, so run one app at a time** and stop it before demonstrating a sibling.

Order: containers up, seed accounts, seed the estate, seed precedent, start the API, start the tower, sign in.

2. Read these first

- 1. [02-hld.md](#), the pipeline and the estate-wide capabilities.
- 2. [03-lld.md](#) section 3, packs as data, which is the idea the whole product turns on.
- 3. [06-adr-log.md](#), especially never-auto-block and the deterministic trio.

3. The five things that will surprise you

- 1. **The lane credentials live in the sibling repository's configuration.** Both projects share one model deployment, so a fresh clone here typically has blank placeholders and every assessment dies instantly with an invalid-address error. It costs nothing, but it looks like a serious failure.
- 2. **Three things must stay plain code:** the risk roll-up, pack applicability, and the compliant-or-flagged decision. Turning any of them into a model call would silently destroy both the benchmark and the free pack-swap demonstration.
- 3. **The tier's capitalisation is normalised exactly once, at fan-in.** Any new reader that touches the raw value will work in some runs and not others.
- 4. **Seeded assets have empty audit chains on purpose.** If you ever "fix" that by seeding chains, the tamper-evidence demonstration becomes meaningless.
- 5. **The labelled benchmark assets deliberately do not appear in the estate.** Adding them would let an inspector retrieve an answer through precedent instead of judging, and the accuracy numbers would stop meaning anything.

4. Where the important logic lives

Question	File
Whether a policy rule applies to an asset	<code>fires()</code> in <code>app/packs.py</code>
How findings become a risk level	<code>risk_rollup()</code> in <code>app/state.py</code>

Question	File
What makes a finding valid	<code>valid_finding()</code> in <code>app/state.py</code>
Which inspectors run for an asset	<code>orchestrate_agent</code> in <code>app/agents.py</code> and the conditional edge in <code>app/graph.py</code>
How a rulebook swap re-scores the estate	<code>rescore_policy()</code> in <code>app/sweep.py</code>
How an inspector thinks and uses tools	<code>app/agents_base.py</code>
How the model lane retries and times out	<code>app/router.py</code>

5. Open work, in the order worth doing

1. **Tier accuracy**, currently 73 percent with at least one asset under-rated. The highest-stakes judgement the system makes.
2. **Endpoint-level tests for the rest of the API**. The flag-decision endpoint has eight, the remediation endpoints nineteen and evidence eleven; registration, packs, sweep and the brief have none. This is possible because table creation moved out of module import into the startup hook.
3. **Evidence retention and withdrawal**. Upload, listing and download shipped 2026-07-29 (ADR-014), with no delete route on purpose. If a file is ever uploaded in error, the answer is a new recorded event that supersedes it, not an erasure; that event does not exist yet. Nothing prunes the `evidence` table either, so a long-running estate grows unbounded.
4. **A test asserting asset descriptions can never reach a third party**.
5. **Signed or version-controlled packs**, with a record of who activated what and when. The rulebook is the trust anchor and today it is a writable file.
6. **Per-team access boundaries** on the estate.
7. **Extract the shared core** with the sibling systems, per the decision recorded in the planning repository.

6. Operational cautions

- `docker compose down -v` destroys the estate, the precedent collection and every audit chain. Re-seed afterwards.
- The compose database password is a long random string; an environment file that guesses the project name will fail authentication.
- The embedding cache is pinned inside the repository on purpose, after a temporary-directory cache corrupted itself and crashed seeding.
- Rehearse demonstrations on the free parts: the estate, the pack swap, the audit trail and the brief cost nothing; registrations cost GPU money.

7. Related repositories

This system is a fork of the contract-review system, which is itself a fork of the ticket-triage system. Several modules are near-identical across all three, and this repository has already been caught running an older copy of one of them. Until the shared package exists, a fix in a shared-looking module must be checked in all three.